Lecture 16 (B15): Continuum-Agent DEQNs — Method Comparison

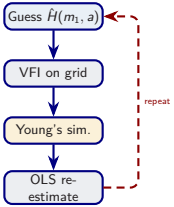
DEQN with histogram encoding, DeepHAM, all-in-one DL, head-to-head

Simon Scheidegger

HEC, University of Lausanne

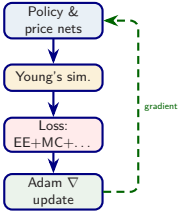
Architecture Comparison: Traditional KS vs. DEQN

Traditional KS



Outer loop; policy sees m_1 only; grid-based VFI.

DEQN + Young's



Single loop; policy sees full histogram; differentiable.

	Traditional KS	DEQN
Distribution info	mean m_1 only	full histogram
Forecasting rule	hand-specified	learned (price net)
Outer loop	implementation-dependent	none
Policy solver	VFI on grid	neural network

Teaching Model Specification (DEQN Version)

Bewley endowment economy (Azinovic et al. 2022, Appendix A.5):

Component	Value
Preferences (EZ)	$\beta = 0.95,\ \rho = 2,\ \sigma = 8$
Asset	Bonds, unit net supply
Idiosyncratic η	$\{0.8, 1.2\},\ \Pi_\eta = \begin{pmatrix} 0.7 & 0.3 \\ 0.3 & 0.7 \end{pmatrix}$
Aggregate shocks	6 states (2×3)
Asset grid	$N_b = 100,\ b \in [0, 20]$
Borrowing	$b_{t+1} \geq 0$

Budget: $c_t + p_t b_{t+1} = b_t + \eta_t w(a_t)$

Aggregate Shocks ($n_Z = 6$)

a_t	Uncert.	Income	$Y(a_t)$
0	Low	Low	0.95
1	Low	Mid	1.00
2	Low	High	1.05
3	High	Low	0.95
4	High	Mid	1.00
5	High	High	1.05

Persistence

= 0.8; product-form transition matrix.

Bridge: From Notebook 10 to Notebook 11

Concept	Notebook 10	Notebook 11
Individual asset	scalar w or k	bond holdings b
Distribution	one histogram $h(k)$	two histograms $(h^{0.8}, h^{1.2})$
Policy input	hand-set rule $g(k)$	NN policy $b'(\eta, b, x_t^{agg})$
One forward step	redistribute mass once	`get_weights_eta()` update
Aggregation	weighted mean	histogram dot product $\mathbf{h}_t \cdot \mathbf{b}'$

Reading order

Read `10_Youngs_Method_Examples` first for the operator, then `11_Continuum_of_Agents_DEQN` for the same operator inside the DEQN state vector and loss.

State Space with Histogram Encoding

Key innovation: encode μ_t as a histogram on N_b grid points.

$$\underbrace{\boxed{\eta_t}\ \boxed{b_t}}_{x_t^{id}\ (2\text{ values})}\quad \underbrace{\boxed{\begin{matrix} z_t^{idx},\ \mathrm{inc}_t^{idx} \\ \mathrm{unc}_t^{idx} \end{matrix}}\ \boxed{h_t^{\eta=0.8}\ (N_b\text{ values})}\ \boxed{h_t^{\eta=1.2}\ (N_b\text{ values})}}_{x_t^{agg}\ (3+2N_b\text{ values})}$$

$$\boxed{\text{Total input dimension} = \underbrace{2}_{\text{individual}} + \underbrace{3+2N_b}_{\text{aggregate}} = 5+2N_b}$$

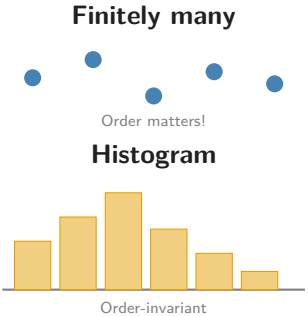
For $N_b = 100$: input dimension $= 5 + 200 =$ **205**.

The aggregate state includes the 6-state shock index, income-level index, uncertainty-regime index, and N_b histogram values for each η type.

Why Histograms? Advantages over Finitely Many Agents

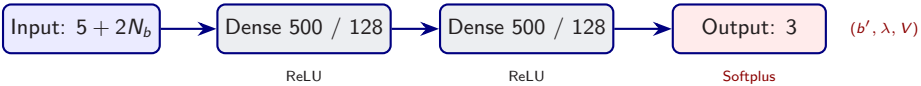
Advantages of histogram encoding

1. **No permutation issue**
   Histogram is order-invariant by construction
2. **Exact aggregation**
   $\int b' \, d\mu = \sum_j h(j) \cdot b'(j)$—no Monte Carlo noise
3. **Direct economic interpretation**
   Network "sees" the full wealth distribution
4. **Scales naturally**
   Grid resolution N_b is a tunable parameter

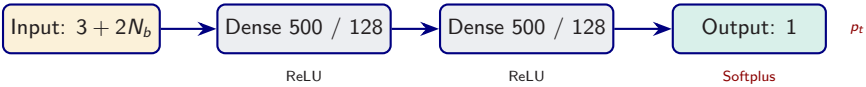


Neural Network Architecture

Policy network $\mathcal{N}_{pol}$:



Price network $\mathcal{N}_{price}$:



- **Policy net:** individual + aggregate state $\rightarrow (b', \lambda, V)$
- **Price net:** aggregate state only $\rightarrow p$
- Softplus keeps outputs strictly positive
- Paper: 500 hidden units. Teaching notebook: 128.
- Both nets are trained jointly on equilibrium residuals

Equilibrium Conditions as Loss

The DEQN solves all equilibrium conditions simultaneously (unsupervised):

1. **Euler equation (EE):** intertemporal optimality condition

$$\text{EE}: \quad \frac{\left(\frac{\beta \cdot \mathbb{E}_t[\cdots] + \lambda_t}{p_t}\right)^{-1/\rho}}{c_t} - 1 = 0$$

2. **Bellman equation (BE):** value function consistency

$$\text{BE}: \quad \frac{V^{opt}}{V^{net}} - 1 = 0$$

3. **Market clearing (MC):** aggregate bond demand = supply

$$\text{MC}: \quad \left| \sum_{\eta} \sum_{j=1}^{N_b} h_t(\eta, b_j) \cdot b'(\eta, b_j, x_t^{agg}) - \bar{B} \right| = 0$$

4. **KKT complementarity:** $\lambda_t \cdot b_{t+1} = 0$
5. **Consumption positivity:** $c_t > 0$ (penalty for violations)

Loss: Euler and Bellman Equations with Epstein–Zin

V^{opt} (Bellman residual: BE $= V^{opt}/V^{net} - 1$)

$$V_t^{opt} = \left[(1-\beta)\, c_t^{1-\rho} + \beta\, \chi_t^{1-\rho}\right]^{\frac{1}{1-\rho}}, \quad \chi_t = \left(\sum_{a',\eta'} \pi_{a'|a}\pi_{\eta'|\eta}\, V_{t+1}^{1-\sigma}\right)^{\frac{1}{1-\sigma}}$$

V_{t+1}: price network at next-period state; c_t from budget constraint.

Euler expectation:

$$\mathbb{E}_t[\cdots] = \sum_{(a',\eta')} \pi_{a'|a}\pi_{\eta'|\eta}\, c_{t+1}^{-\rho} \left(\frac{V_{t+1}}{\chi_t}\right)^{\rho-\sigma}$$

Relative Euler error (sums over all $6 \times 2 = 12$ next-period shock combinations):

$$\boxed{\mathrm{EE} = \frac{\left(\frac{\beta \cdot \mathbb{E}_t[\cdots] + \lambda_t}{p_t}\right)^{-1/\rho}}{c_t} - 1}$$

Loss: Market Clearing via Histogram

Key: Young's histogram enables exact aggregation. No Monte Carlo needed.

$$\boxed{\text{MC} = \left| \sum_{\eta \in \{0.8, 1.2\}} \sum_{j=1}^{N_b} h_t(\eta, b_j) \cdot b'(\eta, b_j, x_t^{agg}) - \bar{B} \right|}$$

- $h_t(\eta, b_j)$: mass at grid point b_j with shock η
- $b'(\eta, b_j, x_t^{agg})$: savings policy from the neural network
- Aggregate demand $= \mathbf{h}_t \cdot \mathbf{b}'$ (dot product; exact, no MC noise)
- Equilibrium: total demand $= \bar{B}$; in the notebook $\bar{B} = 1$ (unit net supply)

Implementation

The policy network is called on all $2N_b$ grid points (η, b_j) with current x_t^{agg}. Aggregate demand $= \mathbf{h}_t \cdot \mathbf{b}'$ is a dot product. Bonds are in net supply $\bar{B}$, normalized to one in the notebook.

Code: `aggdem = tf.reduce_sum(h * b_prime_grid, axis=1)` ≈ 1.

Combined Loss Function

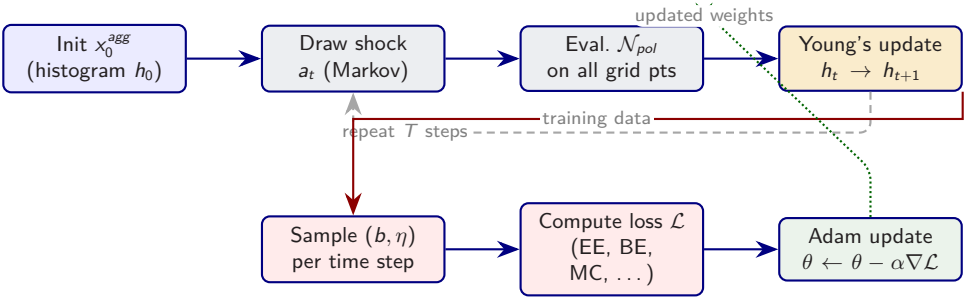
Total loss (sum of squared residuals):

$$\boxed{\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}\left[\mathrm{EE}_i^2 + \mathrm{BE}_i^2 + (n_Z \cdot \mathrm{MC}_i)^2 + \mathrm{KKT}_i^2 + \mathrm{CB}_i^2\right]}$$

Term	Condition	Interpretation
EE^2	Euler equation	Intertemporal optimality
BE^2	Bellman equation	Value function consistency
MC^2	Market clearing	Bond market equilibrium
KKT^2	$\lambda \cdot b' = 0$	Borrowing constraint complementarity
CB^2	$c > 0$	Budget feasibility penalty

- All residuals should be zero in equilibrium
- n_Z scaling on MC: ensures comparable magnitude across terms
- Minimizing $\mathcal{L}$ finds an approximate competitive equilibrium

Young's Method Inside the DEQN Training Loop



The histogram evolves during training: the network adapts to the equilibrium distribution.

Distribution Update in TensorFlow

```python
def get_weights_grid(asearch, agrid, deltaa):
    """Interpolation weights for Young's method."""
    dif = tf.abs(agrid - asearch)
    mask = tf.cast(dif < deltaa, tf.float32)
    w = (1.0 - dif / deltaa) * mask
    w = w / tf.reduce_sum(w, axis=1, keepdims=True)
    return w

def get_weights_eta(net_pol, X_agg):
    """Full histogram update using policy network."""
    h_new = tf.zeros_like(h)
    for eta_idx in range(n_eta):
        b_next = net_pol(eta, grid, X_agg)[:, 0:1]
        weights_next = get_weights_grid(b_next, agrid, deltaa)
        for eta_next in range(n_eta):
            h_new[eta_next] += pi_eta[eta_idx, eta_next] \
                               * (h[eta_idx] @ weights_next)
    h_new = normalize_per_eta(h_new, nA, target_mass=0.5)
    return h_new
```

`get_weights_grid`: interpolation weights.

`normalize_per_eta`: keeps each η block at mass 0.5.

Episode-Based Training

Training Algorithm

Input: Networks $\mathcal{N}_{pol}$, $\mathcal{N}_{price}$; Adam optimizer ($\alpha = 10^{-5}$)
Initialize histogram: equal mass at a single grid point
for episode $= 1, \ldots, N_{\text{ep}}$ **do**

1. Simulate aggregate shocks for T steps
2. Propagate histogram via Young's method using current $\mathcal{N}_{pol}$
3. For each t: sample random (b, η) pairs as idiosyncratic states
4. Form training batch: $\{(x_t^{id}, x_t^{agg}, x_{t+1}^{agg})\}$
5. Train for 1 epoch (batch size $= 128$, Adam)
6. Update starting histogram from end of simulation

end for

Notebook modes: smoke ($N_{\text{ep}} = 100$, $T = 256$), classroom (500, 256), production (65,000, 2048).

Training Design Choices

Why episode-based training?

- The policy network changes the next histogram
- The next histogram changes prices and market clearing
- So the distribution must co-evolve with the network

Practical implementation

- Default notebook mode is smoke so the full pipeline runs during class
- Carry the ending histogram into the next episode
- Use several parallel tracks for diverse starting points
- Check diagnostics on actual simulated states, not schematic plots

How to Evaluate a Real Training Run

Do not use schematic plots

Diagnostics should come from actual notebook output or from paper-reported tables and figures, not from hand-drawn convergence shapes.

1. Residuals

- Euler residuals
- Market-clearing residuals
- KKT / Bellman terms

2. Policy checks

- Monotonicity in wealth
- Borrowing limit respected
- Sensible response to income

3. Distribution checks

- Histogram remains normalized
- Mean evolves plausibly
- Prices clear given the histogram

Why Young's Method Matters Inside DEQNs

The role of Young's method in a DEQN

1. Finite-dimensional encoding of the distribution.
2. Differentiable forward simulation (backprop-compatible).
3. Exact market clearing computed from the histogram dot product.
4. The policy network is evaluated on the full $2 \times N_b$ grid, not just on sampled training states.

Takeaway

The DEQN does not replace the distributional law of motion. Young's method remains the device that turns individual decisions into an aggregate state that can be differentiated through.

Where Are We?

1. Motivation: Why Track Distributions? ✓
2. Young's (2010) Non-Stochastic Simulation ✓
3. KS benchmark + teaching-model switch ✓
4. DEQN teaching implementation ✓
5. $\Rightarrow$ Alternative deep-learning approaches to KS

Part V

Alternative Deep-Learning Approaches to Krusell–Smith

Three Deep-Learning Recipes for KS

All three recipes minimize residuals of the *same* structural equations via SGD. They differ in how the aggregate distribution is fed to the network.

	Histogram DEQN (this lecture so far)	**All-in-one DL** Maliar–Maliar–Winant	**DeepHAM** Han–Yang–E
Distribution input	fixed-grid histogram vector	explicit N-agent panel	learned generalized moments
Input dimension	$\mathcal{O}(N_b)$	$\mathcal{O}(N)$	$M \ll N_b$ (learned)
Permutation invariance	automatic	approximate (LLN)	architectural (DeepSets)
Main reference	Azinovic et al. (2022)	Maliar, Maliar & Winant (2021)	Han, Yang & E (2023)

The accompanying notebook `12_KrusellSmith_DeepLearning.ipynb` implements the middle column at classroom scale.

All-in-One Deep Learning (Maliar, Maliar & Winant 2021)

Idea. Cast every equilibrium condition as a *residual that should vanish*, and train deep networks by minimizing the squared residual on simulated paths.

Loss components (for Krusell–Smith)

- **Euler residual:** $\mathrm{EE} = u'(c_t) - \beta\,\mathbb{E}[u'(c_{t+1})R_{t+1}]$, Monte Carlo over next-period shocks.
- **Bellman residual** on an auxiliary value network.
- **Market clearing:** $\sum_{i=1}^{N} k_{t+1}^{i} = N\bar{K}_{t+1}$.
- **Borrowing constraint:** softplus on savings head or Fischer–Burmeister residual.

Reference: Maliar, Maliar & Winant (2021), *J. Monet. Econ.* 122, 76–101.

All-in-One DL: Scale and Replication

Scale reported by the authors.

- Up to $N = 10{,}000$ explicit agents.
- State dimension 2,001+ (one per agent plus aggregate).
- Approximation errors below 1% on consumption-saving.

Replication code.
`github.com/marcmaliar/deep-learning-euler-method-krusell-smith`

In this course. Notebook `12_KrusellSmith_DeepLearning.ipynb` is a classroom-scale caricature of this approach: single policy network, panel of $N \sim 100$ agents, CPU-friendly, $\sim$10 min to convergence.

DeepHAM — Learned Moments (Han, Yang & E)

Idea. Instead of tracking the histogram or committing to the first moment *a priori*, learn which summary statistics agents actually need.

Generalized moments via DeepSets

$$\boldsymbol{m}_t = \left(\sum_i g_\theta^1(s_t^i),\ \ldots,\ \sum_i g_\theta^M(s_t^i) \right), \qquad M \ll N_b$$

Each $g_\theta^m : \mathbb{R}^d \to \mathbb{R}$ is a neural encoder trained *jointly* with the value and policy networks. Sum-pooling $\Rightarrow$ permutation invariance.

Reference: Han, Yang & E (2024), *Quantitative Economics* (forthcoming); preprint arXiv:2112.14377.

DeepHAM: Reported Accuracy and Extensions

Results on baseline KS:

- First moment only: Bellman error reduced by 37.5% vs. classical KS.
- With one *learned* moment: Bellman error reduced by 54.2%.

Extensions where DeepHAM shines.

- Multiple endogenous states (capital + bonds).
- Large aggregate shocks (risky steady state).
- Zero lower bound; occasionally binding constraints that bite on a non-trivial mass.

These are settings where both classical KS and the local perturbation method of Reiter (2009) break down.

Head-to-Head: When to Use Which?

	Pros	When it shines
Histogram DEQN (Azinovic et al., 2022)	Interpretable state; exact market clearing; DEQN-template reuse	Teaching; N_b moderate; smooth policies
All-in-one DL (Maliar, Maliar & Winant, 2021)	No grid design; large-N panels; single optimizer for all residuals	Large-N research problems; OLG-many-cohort extensions
DeepHAM (Han, Yang & E, 2023)	Learned moments; risky steady state; ZLB; multiple endogenous states	Rich HA macro-finance with nonlinear aggregate effects

Classical baseline

The canonical traditional KS implementation (`econ-ark/KrusellSmith` REMARK) achieves $R^2 > 0.9999$ in ~ 20 outer iterations on the baseline model. All deep-learning methods aim to match or beat this, and to extend beyond the benchmark where KS can't.

Replication Codes + Where to Get Them

Approach	Repository (on GitHub)
Traditional KS (baseline)	`econ-ark/KrusellSmith` *REMARK replication; forecasting-rule algorithm.*
All-in-one DL (MMW 2021)	`marcmaliar/deep-learning-euler-method-krusell-smith` *TensorFlow implementation accompanying the JME paper.*
Histogram DEQN	`sischei/DeepEquilibriumNets` *Azinovic, Gaegauf & Scheidegger (2022) code.*
Classroom MMW-style (this course)	`day4/code/12_KrusellSmith_DeepLearning.ipynb` *Self-contained, CPU-friendly, ∼10 min training.*
DeepHAM	Authors' page; contact the authors for code.

Summary

Key Messages

1. **HA models** require tracking the wealth distribution, an infinite-dimensional state variable.
2. **Young (2010)** gives a deterministic histogram update: no Monte Carlo noise, exact mean preservation.
3. **Krusell–Smith** is the benchmark layer: it explains approximate aggregation and the forecasting-rule loop.
4. **Notebook 11** is the teaching-code layer: Appendix A.5 reuses Young's method inside a DEQN with policy and price networks.
5. **Young + DEQN** keeps exact histogram aggregation while removing the separate KS regression step.

Core References

- Azinovic, M., Gaegauf, L., & Scheidegger, S. (2022). Deep equilibrium nets. *Int. Econ. Rev.* 63(4), 1471–1525.
- Den Haan, W. (2010). Assessing the accuracy of the aggregate law of motion in models with heterogeneous agents. *J. Econ. Dyn. Control* 34(1), 79–99.
- Han, J., Yang, Y., & E, W. (2024). DeepHAM: A global solution method for HA models with aggregate shocks. *Quant. Econ.* (forthcoming); arXiv:2112.14377.
- Krusell, P., & Smith, A.A. (1998). Income and wealth heterogeneity in the macroeconomy. *J. Polit. Econ.* 106(5), 867–896.
- Maliar, L., Maliar, S., & Winant, P. (2021). Deep learning for solving dynamic economic models. *J. Monet. Econ.* 122, 76–101.
- Maliar, L., Maliar, S., & Valli, F. (2010). *J. Econ. Dyn. Control* 34(1), 42–49.
- Reiter, M. (2009). Solving heterogeneous-agent models by projection and perturbation. *J. Econ. Dyn. Control* 33(3), 649–665.
- Young, E.R. (2010). Solving the incomplete markets model with aggregate uncertainty . . . *J. Econ. Dyn. Control* 34(1), 36–41.

Bold-face entries are the two alternative DL methods covered in Part V and in the reading PDFs in `day4/readings/`.

Discrete-time DEQN Notebook Map

Core (Parts II–IV)

`10_Youngs_Method_Examples`

`11_Continuum_of_Agents_DEQN`

Sequence-space extension

`05_SequenceSpace_BrockMirman`

`06_SequenceSpace_KrusellSmith`

Alternative DL (Part V)

`12_KrusellSmith_DeepLearning`
All-in-one DL; classroom scale

External reference code

`econ-ark/KrusellSmith`

`marcmaliar/deep-learning-euler-method-krusell-smith`

All notebooks are on Nuvolos Cloud and the course GitHub repository.

Questions?

Simon Scheidegger

HEC, University of Lausanne

https://sischei.github.io

Materials:

https://github.com/sischei/Deep_Learning_for_Solving_And_Estimating_Dynamic_Economic_Models

Next up — Toolkit and continuous-time block:

Toolkit T1: Agentic Programming

Using AI agents for research

PINNs & continuous-time HA

Surrogates & GPs

Climate & Deep UQ